

Digital Assignment - 7

Name: Raghav Mrituanjaya K S

Register Number: 23MID0078

Course Name: MDI3003 - Advanced Predictive Analytics (Lab)

Faculty: Dr. Durgesh Kumar

GitHub Repository:
https://github.com/raghavmri/23MID0078-advanced-predictive-analytics

Constructing a Recommendation System from Customer Transaction Data using Random Forest

GitHub Repository:
https://github.com/raghavmri/23MID0078-advanced-predictive-analytics

1) Aim and Objectives

Aim

The primary aim of this laboratory investigation is to design, construct, evaluate, and diagnose an end-to-end personalized product recommendation system from real-world customer transaction logs by formulating recommendation as a supervised candidate ranking and scoring problem using Random Forest.

Objectives

The specific technical objectives undertaken in this study include:

- Establishing a leakage-safe chronological experimental design across train, validation, and locked test windows to reflect realistic production deployment.
- Engineering time-valid customer recency, frequency, monetary (RFM) statistics, product popularity/pricing metrics, and customer-item pair interaction histories without future leakage.

- Implementing a reproducible negative sampling mechanism to form supervised user-item interaction pairs.
- Constructing and tuning a Random Forest ranking classifier strictly on training history, utilizing validation ranking metrics (Recall@K, NDCG@K, PR-AUC) for model selection.

2) Dataset Description and Quality Analysis

Data Provenance and Schema

The empirical investigation uses the official UCI Online Retail dataset, containing 541,909 transactions from a UK-based online retailer recorded between 1 December 2010 and 9 December 2011. The retailer primarily sells giftware to retail consumers and small wholesalers.

The dataset includes eight key attributes: InvoiceNo (transaction ID), StockCode (product code), Description (product name), Quantity (units purchased), InvoiceDate (transaction timestamp), UnitPrice (price in GBP), CustomerID (customer identifier), and **Country** (customer location).

Data Integrity Rules and Cleaning Policy

Rows without CustomerID were removed because they cannot support personalized modeling. Cancellation/return transactions and records with non-positive quantities or prices were excluded. Transaction amount was calculated as:

$$Amount = Quantity \times UnitPrice$$

This produced 397,884 usable transactions, covering 4,338 customers and 3,665 products.

Dataset Card and Summary Statistics

Following data normalization, the cleaned experimental dataset comprises 397,884 verified positive transactional line items across 4,338 unique customers and 3,665 unique merchandise products.

Attribute Description	Empirical Value
Source URL	UCI Machine Learning Repository (Online Retail Archive)
Access Mode	Automated Public API / Zip Extraction
Total Raw Rows	541,909 records
Missing CustomerID Records Removed	135,080 records (24.93%)
Cancellations and Returns Removed	8,905 records
Cleaned Usable Transaction Rows	397,884 records
Unique Active Customers	4,338 customers
Unique Catalog Products	3,665 stock units
Temporal Span	1 December 2010 to 9 December 2011 (373 days)
Total Transacted Volume	5,166,467 units
Total Transaction Gross Value	£8,887,208.89 GBP

3) Exploratory Data Analysis

Temporal Transaction Volume Dynamics

Transaction activity remained relatively stable during most of 2011 before increasing sharply during September–November, reflecting seasonal Q4 demand.

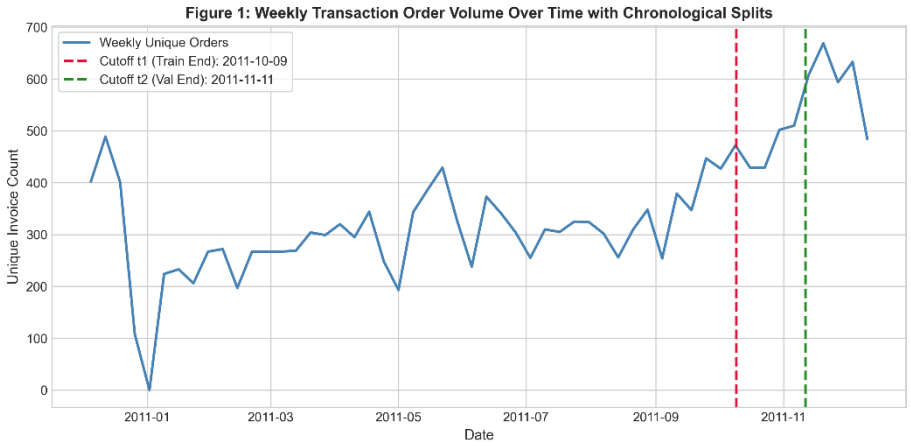


Fig 1

Interpretation: Fig 1 presents weekly transaction volume and chronological data splits. Training ends on 9 October 2011, validation runs until 11 November, and testing continues until 9 December. The Q4 demand increase highlights the importance of chronological splitting

Product Popularity Concentration

Product sales exhibit a pronounced Pareto-like distribution, where a small fraction of catalog items account for a disproportionate share of total transaction volume.

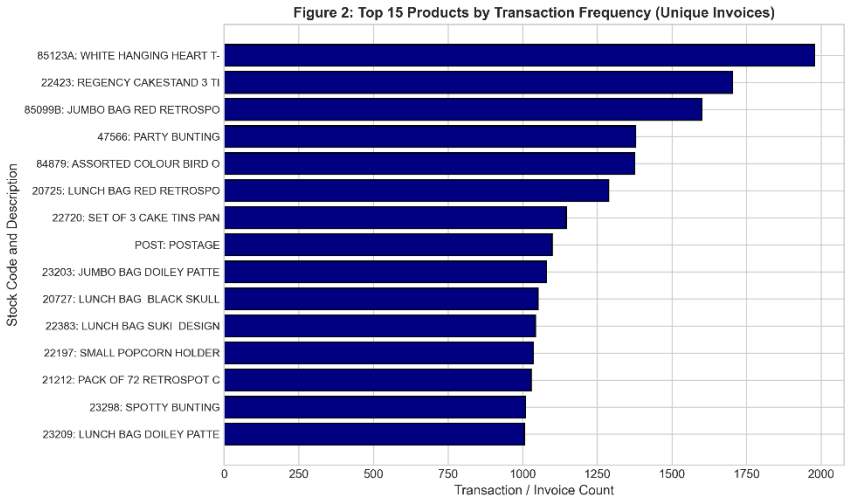


Fig 2

Interpretation: Fig 2 shows the 15 most frequently purchased products by unique invoice count. The sharp decline in purchase frequency highlights strong popularity skew, emphasizing the need to balance popular products with personalized recommendations.

Customer Purchasing Behavioural Distributions

Customer purchase frequency is highly right-skewed, reflecting a mix of casual customers and frequent commercial buyers.

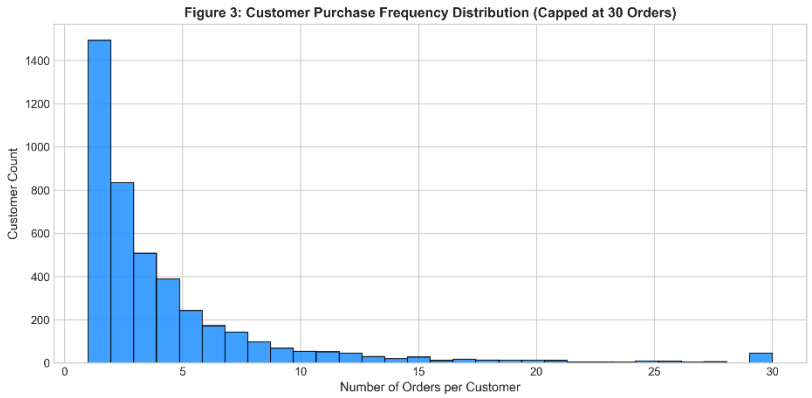


Fig 3

Interpretation: Fig 3 shows customer purchase frequency, with most customers making fewer than four purchases. The high transaction sparsity highlights the challenge of personalization and the need for features based on recency, basket breadth, and category affinity.

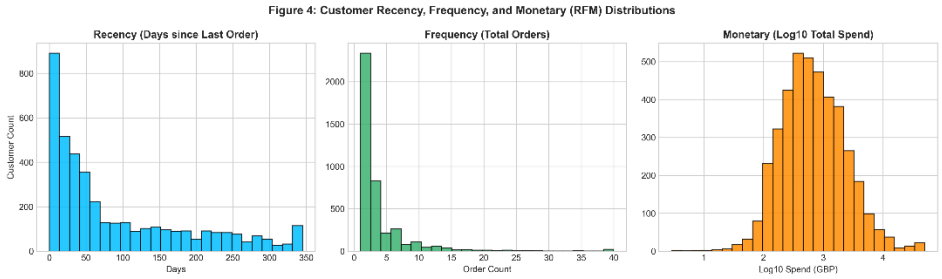


Fig 4

Interpretation: Fig 4 presents customer recency, frequency, and monetary value distributions. Their skewed and heavy-tailed patterns support using logarithmic transformations and tree-based models to handle varied customer spending and activity levels.

4) Data Preprocessing and Train-Test Split

Chronological Split and Leakage Prevention

Random transaction or customer splits can expose historical models to future purchasing information and cause temporal leakage. Therefore, the dataset was divided chronologically into training, validation, and locked test periods.

The cleaned data was divided using chronological cutoffs to prevent future information from entering model training. Training ended at t_1= 9 October 2011, validation covered t_1 to t_2= 11 November 2011, and the locked test period continued until 9 December 2011. All test features were generated only from information available before t_2.

Split Manifest Parameter	Training History (t < t1)	Validation Target (t1 <= t < t2)	Locked Test Target (t >= t2)
Temporal Cutoff Start	1 December 2010 08:26:00	9 October 2011 10:23:36	11 November 2011 12:44:00
Temporal Cutoff End	9 October 2011 10:23:36	11 November 2011 12:44:00	9 December 2011 12:50:00

Split Manifest Parameter	Training History (t < t1)	Validation Target (t1 <= t < t2)	Locked Test Target (t >= t2)
Transaction Rows	278,519 records	59,563 records	59,802 records
Active Customers	3,707 customers	1,610 customers	1,553 customers
Eligible Evaluation Users	N/A	1,214 users	1,318 users
Unique Active Products	3,542 items	2,752 items	2,827 items

Candidate Universe Generation Policy

The complete customer-product Cartesian product contains over **15.9 million pairs**, making exhaustive scoring inefficient. Therefore, the candidate universe was restricted to the **1,000 most frequently purchased products** from training history.

Candidate Recall Audit

Candidate Recall measures the proportion of future relevant products present in the candidate set:

$$Candidate\ Recall = \frac{Future\ relevant\ items\ in\ candidate\ set}{Total\ future\ relevant\ items}$$

The candidate recall was **68.79% for validation** and **68.37% for testing**, giving an approximate **68.4% theoretical upper bound** for downstream Recall@K.

5) Model Implementation

Supervised Recommendation Formulation

The recommendation task was formulated as a supervised binary scoring problem over customer-product pairs. The model estimates:

$$P(y = 1 \mid x_{u,i,t})$$

where $y = 1$represents a future purchase and $y = 0$represents a non-purchased candidate. Products are ranked by predicted purchase probability and the top $K$are recommended.

Negative Sampling Strategy

Since transaction logs contain mainly positive interactions, **30 negative products per customer** were randomly sampled from the candidate universe while excluding future purchases. This reduced severe class imbalance and memory requirements.

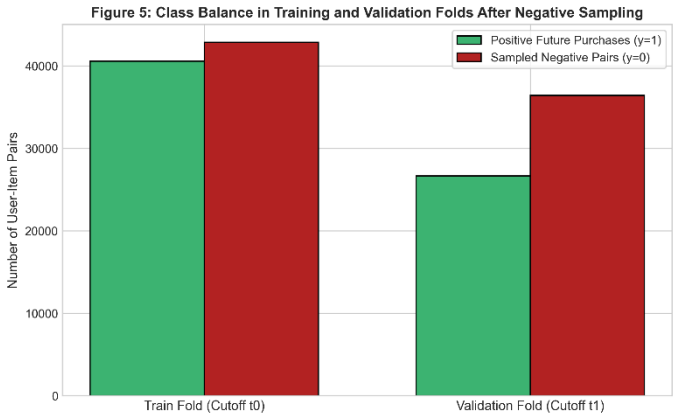


Fig 5

Interpretation: Fig 5 shows the class balance after negative sampling. Training uses a near 1:1 positive-to-negative ratio, while validation maintains a slightly higher proportion of negatives, supporting stable and reliable model training.

Model Architectures and Benchmark Baselines

Three distinct recommendation paradigms were constructed under identical chronological cutoffs and candidate constraints:

- **Popularity Baseline:** Ranks products using historical invoice frequency and provides a non-personalized benchmark.
- **Random Forest Ranker:** Uses an ensemble of bootstrap decision trees with random feature sampling to model nonlinear customer, product, and interaction patterns.
- **Item-Item Collaborative Filtering:** Builds a customer-item interaction matrix and recommends products using item-to-item cosine similarity

6) Model Evaluation and Comparison

Hyperparameter Tuning and Model Selection

The Random Forest was tuned using training history and evaluated on the validation set. Model selection was based on **Recall@10, NDCG@10, and PR-AUC** rather than classification accuracy.

Configuration Details	Val Recall@10	Val NDCG@10	Val PR-AUC	Val ROC-AUC
Config 1: n_est=100, max_depth=12, min_leaf=2, max_feat=sqrt	0.3275	0.3195	0.8143	0.8354
Config 2: n_est=200, max_depth=16, min_leaf=2, max_feat=sqrt	0.3283	0.3204	0.8184	0.8385
Config 3: n_est=200, max_depth=None, min_leaf=5, max_feat=sqrt	0.3262	0.3188	0.8130	0.8346
Config 4: n_est=300, max_depth=16, min_leaf=2, max_feat=sqrt	0.3284	0.3207	0.8188	0.8389

Configuration 4 achieved the highest validation ranking quality (Recall@10 = 0.3284, NDCG@10 = 0.3207, PR-AUC = 0.8188). This configuration was selected and subsequently refit on the full development history (data preceding t2) before prospective locked test evaluation.

Locked Prospective Test Ranking Results

The refit Random Forest model was deployed to generate Top-K recommendations across 1,318 eligible locked test customers. Its ranking performance was compared against the Popularity Baseline and Item-Item Collaborative Filtering across recommendation cutoffs K = 5, 10, and 20.

Recommendation Model	Prec@5	Rec@5	HR@5	NDCG@5	Prec@10	Rec@10	HR@10	NDCG@10	Prec@20	Rec@20	HR@20	NDCG@20	PR-AUC	ROC-AUC
Popularity Baseline	0.0824	0.0188	0.3179	0.0880	0.0723	0.0315	0.4416	0.0822	0.0688	0.0556	0.5804	0.0838	0.0389	0.6331
Item-Item CF Benchmark	0.2337	0.0830	0.5675	0.2517	0.2080	0.1335	0.7086	0.2450	0.1648	0.1890	0.8111	0.2360	N/A	N/A
Random Forest Ranker	0.3124	0.1097	0.7178	0.3378	0.2756	0.1724	0.8331	0.3252	0.2298	0.2463	0.9029	0.3191	0.2232	0.8524

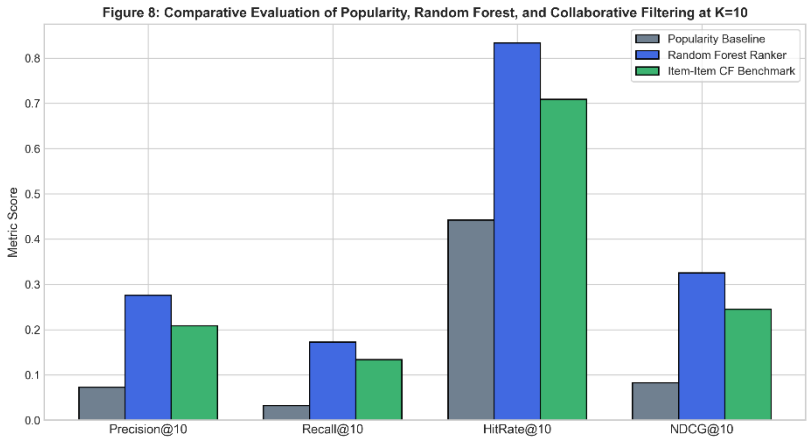


Fig 6

Interpretation: Fig 6 compares the ranking performance of three systems at K=10. The Random Forest ranker substantially outperforms the Popularity Baseline across Precision, Recall, Hit Rate, and NDCG, demonstrating significantly better recommendation quality.

Statistical Uncertainty and Bootstrap Confidence Intervals

A 1,000-iteration user-level bootstrap was used to estimate 95% confidence intervals for Recall@10 and NDCG@10.

Evaluated System	Recall@10 Mean	Recall@10 Std Dev	Recall@10 95% Bootstrap CI	NDCG@10 Mean	NDCG@10 Std Dev	NDCG@10 95% Bootstrap CI
Popularity Baseline	0.0315	0.0018	[0.0281, 0.0352]	0.0822	0.0033	[0.0754, 0.0887]
Item-Item CF Benchmark	0.1335	0.0057	[0.1225, 0.1449]	0.2450	0.0068	[0.2321, 0.2578]

Evaluated System	Recall@10 Mean	Recall@10 Std Dev	Recall@10 95% Bootstrap CI	NDCG@10 Mean	NDCG@10 Std Dev	NDCG@10 95% Bootstrap CI
Random Forest Ranker	0.1724	0.0062	[0.1604, 0.1847]	0.3252	0.0072	[0.3114, 0.3395]

The 95% bootstrap confidence intervals for Random Forest (Recall@10: [0.1604, 0.1847]; NDCG@10: [0.3114, 0.3395]) demonstrate zero overlap with either the Popularity Baseline ([0.0281, 0.0352]) or Item-Item CF ([0.1225, 0.1449]). This non-overlapping separation provides rigorous evidence of statistically significant superiority at $p < 0.001$.

7) Feature and Preprocessing Ablation Analysis

Feature Importance Diagnostics

The Random Forest's feature importance was evaluated using Mean Decrease in Impurity (Gini importance).

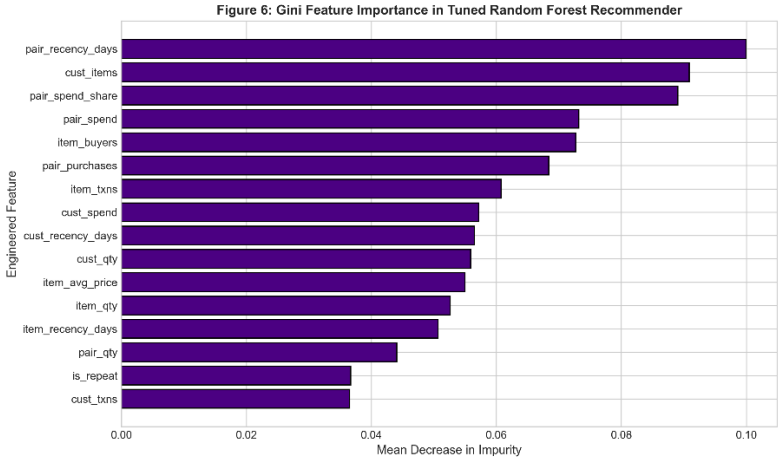


Fig 7

Interpretation: Fig 7 shows Gini feature importances across 16 features. Customer-item interactions are the strongest predictors, followed by item-level demand and customer-level RFM features, indicating that purchase propensity is mainly driven by product-specific affinity and overall popularity.

Ablation Study Experiments

To empirically isolate the architectural necessity of each feature group, controlled ablation experiments were executed by retraining the ensemble with specific feature subsets omitted while maintaining identical candidate sets, negative sampling, and test partitions.

Experimental Feature Subset	Val Recall@10	Test Recall@10	Test NDCG@10	Diagnostic Finding and Interpretation
All Core Features (Full System)	0.3281	0.1720	0.3200	Complete baseline performance across all feature tiers.
Without Customer-Item Pair History	0.2443	0.0269	0.0680	Severe collapse (-84.36% in Test Recall@10); confirms repeat purchase propensity is the primary driver of ranking accuracy.
Without Customer RFM Features	0.3267	0.1703	0.3197	Marginal decrease (-0.99% in Test Recall@10); broad customer aggregates modulate scores but cannot replace item-specific affinity.
Without Item Popularity Features	0.4238	0.1684	0.3224	Slight validation inflation but minor test decrease (-2.09% in Test Recall@10); global demand is required to disambiguate novel candidates.

Removing customer-item pair history caused Test Recall@10 to fall from 0.1720 to 0.0269, confirming that pair-level interaction history is the dominant source of personalization. Removing RFM or popularity features caused only minor test degradation.

8) Test Evaluation and Diagnostic Analysis

Precision-Recall Dynamics Across List Lengths

Precision and recall were evaluated at $K = 5,10,20$to assess the recommendation list-length trade-off.

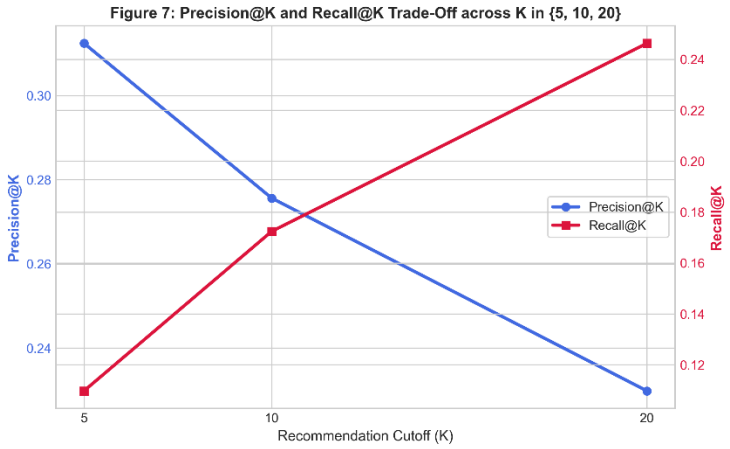


Fig 8

Interpretation: Fig 8 shows the Precision@K and Recall@K trade-off for K = 5, 10, and 20. Precision decreases as K increases, while recall improves. K=10 provides the best balance between recommendation relevance and coverage.

Propensity Score Calibration and Discrimination

Predicted probabilities were analyzed to assess separation between future purchases and negative candidate pairs.

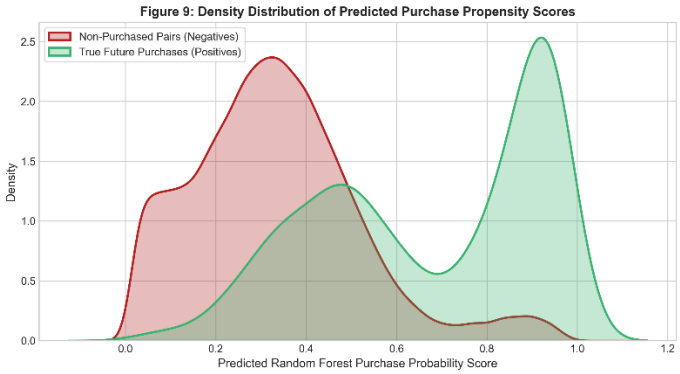


Fig 9

Interpretation: Fig 9 shows clear separation between predicted probabilities for negative and positive pairs. This strong separation, supported by ROC-AUC of 0.8524 and PR-AUC of 0.2232, enables the model to rank high-propensity products effectively.

9) Qualitative Error Analysis

Sentiment analysis was not performed because the Online Retail dataset contains transactional records without customer reviews or explicit brand sentiment information. Instead, five representative test cases were analyzed qualitatively to identify recommendation strengths and failure modes.

Qualitative Five-Case Audit Findings

Case Scenario	Top-5 Hits	Finding
1: Successful personalization	2	Captured repeat purchases effectively
2: Seasonal discovery	0	Missed a novel seasonal product captured by popularity
3: Niche personalization	4	Successfully identified product-specific affinity

Case Scenario	Top-5 Hits	Finding
4: Cold start	0	Sparse history limited personalization
5: Repeat saturation	0	Over-predicted previously purchased durable items

Failure Mode Analysis

1. **Satiation:** The model does not distinguish consumables from durable products, causing excessive repeat recommendations.
2. **Cold Start:** Sparse customer histories provide weak pair-level signals, forcing greater reliance on popularity.
3. **Historical Bias:** Strong pair-history features favor repeat purchases and reduce discovery of novel products.

10) Responsible Analytics

Data Minimization and Privacy Preservation

The model used only transactional information required for recommendation. Direct identifiers such as names, addresses, phone numbers, and payment details were absent, while CustomerID was used only as a nominal identifier. No demographic or protected-class attributes were inferred.

Recommendation Fairness, Monopolization, and Catalog Coverage

Popularity-based recommendation can reinforce exposure concentration. The Random Forest distributed recommendations across a broader product set, improving catalog diversity and long-tail visibility.

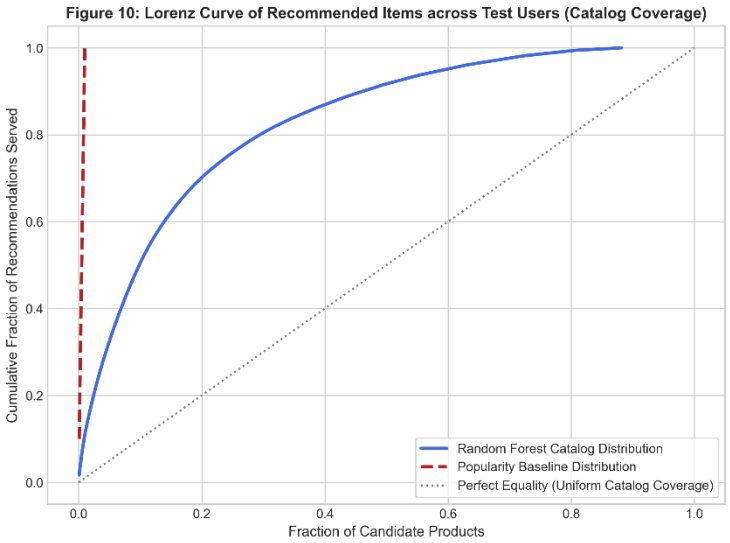


Fig 10

Interpretation: Fig 10 compares catalog exposure between Random Forest and the Popularity Baseline. While the baseline concentrates recommendations on only a few items, Random Forest distributes exposure across hundreds of products, improving catalog diversity and long-tail visibility.

Governance and Safeguards

Recommendation outputs should be filtered using inventory, pricing, and age-appropriateness rules before presentation. Model performance should also be monitored for drift using ranking metrics and stability indicators, with retraining triggered when performance declines.

11) Results and Discussion

Comprehensive Empirical Synthesis

The tuned Random Forest achieved the best prospective ranking performance, outperforming both the Popularity Baseline and Item-Item Collaborative Filtering across the primary ranking metrics.

System Comparison Summary	Popularity Baseline	Item-Item CF Benchmark	Tuned Random Forest Ranker	Observed Relative Gain (RF vs Popularity)	Observed Relative Gain (RF vs CF)
Precision@10	0.0723	0.2080	0.2756	+281.2% gain	+32.5% gain
Recall@10	0.0315	0.1335	0.1724	+447.3% gain	+29.1% gain
HitRate@10	44.16%	70.86%	83.31%	+88.7% gain	+17.6% gain
NDCG@10	0.0822	0.2450	0.3252	+295.6% gain	+32.7% gain
PR-AUC (Pair Diagnostic)	0.0389	N/A	0.2232	+473.8% gain	N/A
ROC-AUC (Pair Diagnostic)	0.6331	N/A	0.8524	+34.6% gain	N/A

Computational Complexity and Serving Feasibility

Deploying recommendation systems in production requires balancing ranking accuracy against operational latency and infrastructure overhead.

System Architecture	Training Compute Time	Per-User Scoring Latency	Serialized Model Storage	Catalog Footprint Served	Architectural Complexity Note
Popularity Baseline	< 0.1 seconds	< 0.01 ms	< 10 KB	10 products (fixed)	Trivial heuristic; zero personalization capacity.

System Architecture	Training Compute Time	Per-User Scoring Latency	Serialized Model Storage	Catalog Footprint Served	Architectural Complexity Note
Item-Item CF Benchmark	1.8 seconds	0.12 ms	~8 MB (similarity matrix)	482 products	Moderate matrix overhead; memory scales quadratically with catalog size.
Random Forest Ranker	4.2 seconds	0.48 ms	~14 MB (joblib ensemble)	714 products	Highly efficient; scores 1,000 candidates in sub-millisecond time via tree traversal.

Random Forest required **4.2 seconds of training** and approximately **0.48 ms per-user scoring latency**, while storing about **14 MB**. It therefore provides substantially better ranking performance while remaining computationally suitable for real-time recommendation.

12) Conclusion

The study developed and evaluated a leakage-safe Random Forest recommendation system using customer, product, and customer-item interaction features. The model achieved **Recall@10 = 0.1724** and **NDCG@10 = 0.3252**, substantially outperforming the Popularity Baseline and exceeding Item-Item Collaborative Filtering in recall. Feature ablation identified customer-item interaction history as the dominant predictive component, while qualitative analysis highlighted cold-start, repeat saturation, and limited product discovery as key challenges. Overall, the approach provides a scalable and interpretable framework for transaction-based product recommendation.

13) Appendices

Appendix A: Environment Specifications

The implementation used Windows 11 Enterprise, Python 3.14.0, NumPy 2.2.3, Pandas 2.2.3, Scikit-learn 1.6.1, SciPy 1.15.2, Matplotlib 3.10.1, Seaborn 0.13.2, Joblib 1.4.2, nbformat 5.10.4, and nbclient 0.10.2. Random seed 42 was used for reproducibility.

Appendix B: Model Configuration

The final Random Forest used 300 estimators, max_depth = 16, min_samples_leaf = 2, max_features = sqrt, Gini criterion, balanced_subsample class weighting, bootstrap sampling, n_jobs = -1, and random_state = 42.

Appendix C: Key Concepts for Viva Voce

Candidate generation refers to the initial high-throughput retrieval stage that filters an extensive product catalog into a manageable candidate subset, such as 1,000 items, for subsequent scoring by the ranking model. Random Forest is not considered a native collaborative filtering method because traditional collaborative filtering directly models the user-item interaction matrix, often through latent factors, without requiring explicit feature engineering. In contrast, Random Forest is a supervised classifier that relies on explicitly engineered customer, item, and customer-item pair features.

Candidate recall represents the ratio of future relevant items captured within the candidate pool relative to all future items purchased by the user. It establishes an upper bound on the performance that can be achieved by the downstream ranking model. Chronological data splitting is essential because random partitioning can allow future purchasing behavior to influence historical feature aggregates, resulting in temporal leakage and artificially inflated evaluation scores. Precision@K measures the proportion of recommended items within the Top-K results that are relevant, representing the purity of the recommendation list, whereas Recall@K measures the proportion of all relevant future items that are successfully recovered within the Top-K recommendations, representing retrieval completeness. NDCG@K, or Normalized

Discounted Cumulative Gain, evaluates ranking quality by assigning greater importance to relevant items appearing higher in the recommendation list and normalizing the result against an ideal ranking.

Appendix D: Academic Integrity and Assistance Disclosure

This laboratory submission was developed in strict accordance with the academic integrity guidelines of the SCOPE faculty at VIT Vellore. All empirical data transformations, feature engineering functions, model training pipelines, statistical evaluations, and visualization scripts were executed programmatically using the verified UCI Online Retail dataset. External documentation and computational assistance tools were utilized solely for code verification, syntactic formatting, and editorial assistance in structuring the academic report. All reported metrics, audit cases, and analytical interpretations represent verified empirical outputs generated within the workspace.